

File Streaming Inbound Endpoint

Every parameter of `org.wso2.carbon.inbound.streaming.VFSConsumer`, its default, and four complete configurations. The inbound reads a file as a stream, so a multi-gigabyte CSV passes through a 1 GB heap without ever being held whole.

Modes ENTIRE_FILE · CHUNK · RECORD

Formats csv · json · jsonl · xml · text

Verified 2.2 GB through a 1 GB heap, peak 404 MB

Contents

- | | |
|---------------------------|--------------------------|
| 1. Declaring parameters | 10. Error handling |
| 2. What one file does | 11. Checkpointing |
| 3. Inbound plumbing | 12. Completion sequences |
| 4. Source and polling | 13. Locking |
| 5. Streaming core | 14. After processing |
| 6. CSV parsing | 15. Remote connections |
| 7. JSON, JSONL, XML, text | 16. Context properties |
| 8. Where output goes | 17. Four configurations |
| 9. Reply file | 18. Gotchas |

1 · Declaring parameters

Every parameter goes inside the `<parameters>` element of an `inboundEndpoint`, deployed to `<MI_HOME>/repository/deployment/server/synapse-configs/default/inbound-endpoints/`.

skeleton.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<inboundEndpoint xmlns="http://ws.apache.org/ns/synapse"
    name="myStreamInbound"
    class="org.wso2.carbon.inbound.streaming.VFSConsumer"
    sequence="mySeq" onError="myFaultSeq" suspend="false">
  <parameters xmlns="http://ws.apache.org/ns/synapse">
    <parameter name="scheduleType">Polling</parameter>
    <parameter name="interval">5000</parameter>
    <!-- ... the rest ... -->
  </parameters>
</inboundEndpoint>
```

MI 4.1.0 ONLY

Use the `class` attribute with `scheduleType`, never `protocol="polling"` — MI 4.1.0 rejects it with `No enum constant ... Protocols.polling`. There is also no `cronExpression` support on this runtime; use `interval`.

2 · What one file does

1. Poll the folder; skip anything not matching `FileNamePattern`.
2. Take the `.lock` beside the file.
3. Look for a checkpoint. *Resumable* → start at record N. *Otherwise* → start at record 1.
4. Read one chunk or record.
5. Inject it into the sequence.
6. Append whatever the sequence left to the reply file.
7. Every `CheckpointInterval` units, write the checkpoint.
8. Repeat from step 4 until the file is exhausted.
9. Clear the checkpoint, run `FileCompleteSequence`, move the file to `MoveAfterProcess`.

A graceful shutdown interrupts the loop at steps 4–8: the inbound finishes the record in flight, writes a checkpoint, releases the lock and leaves the file where it is. Neither completion sequence fires — the file is paused, not finished.

3 · Inbound plumbing

Synapse-level parameters, not specific to this inbound.

PARAMETER	DEFAULT	WHAT IT DOES
<code>scheduleType</code> REQ	–	Must be <code>Polling</code> . Pairs with the <code>class</code> attribute.
<code>interval</code> REQ	–	Milliseconds between polls of the source folder.
<code>sequential</code>	<code>true</code>	Whether polls run one at a time. Streaming always delivers records sequentially within a file whatever this is set to; it only affects concurrency across polls.
<code>coordination</code>	<code>true</code>	In a cluster, only the coordinator polls. Leave it on, or several nodes will fight for the same file.

The `sequence` attribute names the sequence each chunk or record is injected into; `onError` names the fault sequence.

4 · Source and polling

Names below omit the `transport.vfs.` prefix.

PARAMETER	DEFAULT	WHAT IT DOES
<code>FileURI</code> REQ	–	The folder (or single file) to poll. A local path, or a VFS URI — <code>sftp://</code> , <code>ftp://</code> , <code>https://</code> , <code>smb2://</code> .
<code>FileNamePattern</code>	<code>all</code>	Java regex a file name must match, e.g. <code>.*\.csv</code> .
<code>ContentType</code>	<code>none</code>	Content type for the whole-file path. Ignored in CHUNK and RECORD modes, where the format fixes it.
<code>fileType</code>	<code>none</code>	VFS file type hint for the connection.
<code>FileProcessCount</code>	<code>all</code>	Maximum files consumed per poll. Set to <code>1</code> for large files so peak memory stays predictable.
<code>FileProcessInterval</code>	<code>0</code>	Milliseconds to pause between files within one poll.
<code>FileSortAttribute</code>	<code>none</code>	Sort key: <code>Name</code> , <code>Size</code> or <code>Lastmodifiedtimestamp</code> .
<code>FileSortAscending</code>	<code>true</code>	Sort direction for the above.
<code>MinimumAge</code>	<code>none</code>	Milliseconds a file must have existed before it is touched. The simplest guard against reading a file still being written.
<code>MaximumAge</code>	<code>none</code>	Files older than this are skipped and left in place.
<code>FileSizeLimit</code>	<code>-1</code>	Bytes; <code>-1</code> is no limit. Larger files are skipped.
<code>CheckSizeInterval</code>	<code>none</code>	Milliseconds between two size readings. The file is only consumed if the size did not change — the robust guard against a partial upload.
<code>CheckSizeIgnoreEmpty</code>	<code>false</code>	Skip zero-byte files silently instead of treating them as failures.
<code>waitTimeBeforeRead</code>	<code>0</code>	Milliseconds to wait after the file is seen before opening it.
<code>ClusterAware</code>	<code>false</code>	Enables cluster-aware polling behaviour.
<code>IsMounted</code>	<code>false</code>	Declares the source as a shared mount, which changes how locks are judged.
<code>Build</code>	<code>false</code>	Build the message on the whole-file path before injection.

PARAMETER	DEFAULT	WHAT IT DOES
ResolveHostsDynamically	false	Re-resolve the host name on each connection instead of caching it.
ReconnectTimeout	30000	Milliseconds before a failed remote connection is retried.
MaxRetryCount	3	Connection attempts before the poll gives up.
CreateFolder	false	Create the destination folders if they do not exist.
UpdateLastModified	false	Touch the file's timestamp when it is moved.
DistributedLock	false	Use a distributed lock instead of the file-based one.
DistributedTimeout	none	Timeout, in seconds, for that distributed lock.

5 · Streaming core

Names below omit the `transport.vfs.` prefix.

PARAMETER	DEFAULT	WHAT IT DOES
Streaming REQ	false	Must be <code>true</code> for CHUNK or RECORD mode. Left at the default, the inbound takes the whole-file path regardless of <code>StreamingMode</code> .
StreamingMode	ENTIRE_FILE	<code>ENTIRE_FILE</code> — one message for the file. No streaming, no checkpointing. <code>CHUNK</code> — a JSON array of <code>StreamingChunkSize</code> records per message. <code>RECORD</code> — one JSON object per message.
StreamingInputFormat	text	How the bytes are parsed: <code>text</code> , <code>csv</code> , <code>json</code> , <code>jsonl</code> or <code>xml</code> .
StreamingChunkSize	1	Records per message in CHUNK mode. 1000–5000 is the usual range; larger means fewer mediations but a bigger live payload.
StreamingBufferSize	8192	Read-buffer bytes. 65536 measurably helps on multi-gigabyte files.
StreamingCharset	UTF-8	Character set used to decode the file, and appended to the content type.

EASY TO MISS

`transport.vfs.Streaming` defaults to `false`. Setting `StreamingMode` alone does nothing — both are needed.

6 · CSV parsing

Names below omit the `transport.vfs.` prefix.

PARAMETER	DEFAULT	WHAT IT DOES
StreamingCsvHasHeader	true	Treat the first line as column names. The header is consumed, not delivered as a record.
StreamingCsvDelimiter	,	Field separator. Only the first character is used.
StreamingCsvQuote	"	Quote character.
StreamingAddHeadersToEachResult	false	<code>false</code> emits a row as an array <code>["1", "Alice"]</code> ; <code>true</code> emits it as an object <code>{"id": "1", "name": "Alice"}</code> . Almost always set this to <code>true</code> .
StreamingCsvDataTypes	strings	JSON array declaring the type of named or positional columns. Alias: <code>transport.vfs.csvDataTypes</code> .

Column data types

CSV carries no types, so without this every field is emitted as a string and a sequence cannot do arithmetic on it. The mapping is declared once on the inbound:

```
<parameter name="transport.vfs.StreamingCsvDataTypes">[
  {"Column Name Or Index":"order_id", "Is Column Name":"Yes", "Data Type":"Integer"},
  {"Column Name Or Index":"unit_price", "Is Column Name":"Yes", "Data Type":"Number"},
  {"Column Name Or Index":"2", "Is Column Name":"No", "Data Type":"Boolean"}
]</parameter>
```

KEY	VALUES	MEANING
Column Name Or Index	string	A header name, or a zero-based column index as a string.
Is Column Name	Yes / No	Yes reads the first key as a header name, No as an index.
Data Type	String · Integer Number · Boolean	The JSON type the value is emitted as.

A value that will not convert is **left as a string** and the column is warned about **once per file**, never once per row — a bad column in a million-row file produces one log line, not a million. Nothing is dropped and nothing fails.

With the mapping above, the row `4,Dave,N/A,7` yields

`{"order_id":4,"name":"Dave","unit_price":"N/A","qty":"7"}`: `order_id` typed, `unit_price` left alone because `N/A` is not a number, `qty` untouched because it was not declared.

7 · JSON, JSONL, XML, text

Names below omit the `transport.vfs.` prefix.

PARAMETER	DEFAULT	WHAT IT DOES
StreamingJsonPath	<code>\$[*]</code>	Selects the repeating element in a <code>json</code> file. Applies only to <code>StreamingInputFormat=json</code> .

The other formats need no parameters of their own:

- `jsonl` — one JSON value per line. Each line is a record; no path needed.
- `xml` — repeating child elements become records.
- `text` — one line is one record. The default format.

For a document shaped `{"meta":{"..."},"orders":[...]}`, the selector `$.orders[*]` makes each element of `orders` one record and ignores `meta`. The default `$[*]` suits a file that is a bare top-level array.

8 · Where output goes

Names below omit the `transport.vfs.` prefix.

PARAMETER	DEFAULT	WHAT IT DOES
StreamingOutputProperty	<code>unset</code>	Name of a Synapse property to put the parsed JSON into. Unset, the parsed content becomes the message body instead.

Read it in a sequence with `get-property(' <name>')`. The value is a JSON **object** in RECORD mode and a JSON **array** in CHUNK mode, as a string.

WHY A PROPERTY, NOT A VARIABLE

MI 4.1.0's Synapse has no message variables. The upstream parameter name `transport.vfs.StreamingOutputVariable` is still accepted as an alias so existing configurations keep working.

9 · Reply file

Names below omit the `transport.vfs.` prefix.

PARAMETER	DEFAULT	WHAT IT DOES
<code>ReplyFileURI</code>	<code>unset</code>	Destination folder for the mediated output. Unset, nothing is written.
<code>ReplyFileName</code>	<code>response.jsonl</code> <code>response.xml</code>	Output file name. The default depends on the payload: JSON lands in <code>response.jsonl</code> , anything else in <code>response.xml</code> .

The payload decides the format, not the input: `JsonUtil.hasJsonPayload()` picks the JSON path, otherwise the message is serialised with the registered message formatter for its content type. **Append is forced, not a choice** — a streaming run writes one file per source file and adds to it on every loop.

KEEP THE PAYLOAD ON ONE LINE

A `<payloadFactory>` whose `<format>` spans several lines writes multi-line entries and breaks JSON Lines. Compact the format onto one line.

An empty body is skipped rather than written as a blank line, and a destination that cannot be opened fails softly: the source file has already been delivered to the sequence, so it is not failed over a reply-file problem.

10 · Error handling

Streaming separates a bad **record** from a bad **file**. A bad record never fails the file.

Names below omit the `transport.vfs.` prefix.

PARAMETER	DEFAULT	WHAT IT DOES
<code>StreamingParseErrorAction</code>	<code>MOVE</code>	<code>MOVE</code> collects records that would not parse; <code>DROP</code> discards them.
<code>StreamingParseErrorFolder</code>	<code>unset</code>	Where the collected parse failures are written.
<code>StreamingMediationErrorAction</code>	<code>MOVE</code>	Same, for records the sequence threw on.
<code>StreamingMediationErrorFolder</code>	<code>unset</code>	Where the collected mediation failures are written.

Failed records are collected as JSON Lines: `input.jsonl` produces `input.parse-failed.jsonl` and `input.mediation-failed.jsonl`. Every file ends with a summary line giving the totals:

```
INFO {StreamInjectHandler} - Streaming completed for orders-1gb.csv:
  processed=8663312, parseFailed=0, mediationFailed=0 (resumed from record 7506000)
```

The whole-file failure parameters (`ActionAfterFailure`, `MoveAfterFailure`) still apply to a file that cannot be opened or read at all.

11 · Checkpointing

A long file that is interrupted resumes near where it stopped instead of re-reading from the start.

Names below omit the `transport.vfs.` prefix.

PARAMETER	DEFAULT	WHAT IT DOES
<code>StreamingCheckpointEnabled</code>	<code>true</code>	Whether a resume point is written at all.
<code>StreamingCheckpointInterval</code>	<code>1000</code>	Units between writes — chunks in CHUNK mode, records in RECORD mode. With <code>ChunkSize=2000</code> , an interval of 50 is a resume point every 100,000 rows.

Only CHUNK and RECORD checkpoint. `ENTIRE_FILE` is a single message, so there is no position to record.

Where it lives

The MI registry — a file-based governance registry, **no database**:

```
gov:/fileStreamingCheckpoints/<inboundName>/<crc32 of the file URI>.json
```

One in-flight checkpoint per source file, plus a `.bak` holding the previous write and a `.meta` sidecar the registry keeps. The `.bak` exists so a crash during the write itself still leaves a usable resume point.

The source URI is **never stored**. The resource is named by a CRC32 of it and the body carries only a SHA-256, because an `sftp://` or `smb2://` URI carries credentials and in a cluster the registry is a shared mount.

What is stored

`b9aca662.json`

```
{
  "schemaVersion": 1,
  "inboundName": "bigOrdersInbound",
  "fileUriHash": "9fc088a9528588977f0a0ddbb34d0ff4e38ca6b476b427b85d3eab5b6f103f65",
  "fileFingerprint": {
    "size": 1073749726,
    "lastModified": 1791264772944,
    "hashedBytes": 65536,
    "algorithm": "CRC32",
    "hash": "5aa6208e"
  },
  "streamingMode": "CHUNK",
  "inputFormat": "csv",
  "configHash": "da5db5",
  "recordsConsumed": 2812000,
  "chunksConsumed": 1406,
  "processedRecords": 2812000,
  "parseFailedRecords": 0,
  "mediationFailedRecords": 0,
  "updatedAt": 1791264811852
}
```

`chunksConsumed` is what keeps chunk numbers counting up after a restart instead of going back to 1. The counters keep the final totals absolute across a resume.

File identity is content, not timestamp

A checkpoint is only used when the file still matches the fingerprint recorded with it: **byte count + CRC32 of the leading 64 KB + the hash budget and algorithm**. The modification time is recorded and printed in diagnostics but is **not compared**.

WHY MTIME IS EXCLUDED

Comparing it broke resume in a cluster. Two nodes reported the same local file's mtime as `1791263539594` and `1791263539000` — one instant, the second truncated to a whole second, because the precision depends on the file system and on how the `FileObject` was resolved. The second node judged the file changed and re-read all of it from record 1. The same comparison also rejected a file moved away and copied back with identical bytes.

The **config hash** is the other interlock, covering format, mode, chunk size, charset, CSV delimiter/quote/header, JSON path, output property and column data types. Change any of them and the old position is meaningless, so the file starts fresh rather than skipping the wrong records.

Why a resume did not happen

A refused resume costs a full re-read, so the reason is always logged.

LOG LINE	LEVEL	MEANING
No streaming checkpoint at <path>...	INFO	Nothing to resume from — the ordinary case for a new file.
Ignoring the streaming checkpoint... because <reason>	WARN	One was found and refused. The reason names the differing fields, e.g. the file changed since the checkpoint was written (size 42->1073749726 ...).
The primary streaming checkpoint... was unreadable	WARN	The main resource was corrupt; the <code>.bak</code> was used.

If checkpointing cannot start at all — no registry, or the file cannot be fingerprinted — the file streams without resume support and a warning is logged. It is never a failure.

Sharing the registry across a cluster

For one node to resume what another started, both must read the same governance registry. Point the second node's `GovRegRoot` at the first node's registry in `<MI_HOME>/repository/deployment/server/synapse-configs/default/registry.xml`:

node2 · registry.xml

```
<registry xmlns="http://ws.apache.org/ns/synapse"
  provider="org.wso2.micro.integrator.registry.MicroIntegratorRegistry">
  <parameter name="cachableDuration">15000</parameter>
  <parameter name="GovRegRoot">file:///shared/node1/registry/governance</parameter>
</registry>
```

DO NOT SYMLINK IT

Symlinking `registry/governance` breaks reads: `MicroIntegratorRegistry` resolves the canonical path and checks it still sits under the configured root, and a symlink fails that test. Writes keep working, which hides the problem. Use `GovRegRoot`.

12 · Completion sequences

Two optional sequences fire **once per file**, after the last record, carrying the file's totals.

Names below omit the `transport.vfs.` prefix.

PARAMETER	DEFAULT	WHAT IT DOES
<code>StreamingFileCompleteSequence</code>	<code>unset</code>	Invoked after a file is fully consumed.
<code>StreamingFileFailureSequence</code>	<code>unset</code>	Invoked when a file fails as a whole.

Both receive a JSON summary as the body and the same values as properties:

```
{"file":"orders-1gb.csv","uri":"file:///data/in/orders-1gb.csv","status":"COMPLETED",
"processed":8663312,"parseFailed":0,"mediationFailed":0}
```

PROPERTY	VALUE
<code>STREAMING_FILE_NAME</code>	Base name of the source file.
<code>STREAMING_FILE_URI</code>	Full URI, with any password masked.
<code>STREAMING_STATUS</code>	<code>COMPLETED</code> or <code>FAILED</code> .
<code>STREAMING_PROCESSED</code>	Records delivered to the sequence.
<code>STREAMING_PARSE_FAILED</code>	Records that would not parse.
<code>STREAMING_MEDIATION_FAILED</code>	Records the sequence threw on.

Neither fires on a shutdown interruption. The file is paused, not finished, and will be resumed on the next poll.

13 · Locking

A `.lock` file beside the source stops two nodes consuming it at once.

Names below omit the `transport.vfs.` prefix.

PARAMETER	DEFAULT	WHAT IT DOES
<code>Locking</code>	<code>enable</code>	<code>enable</code> or <code>disable</code> . Keep it on for any shared or clustered source.
<code>AutoLockRelease</code>	<code>false</code>	Reclaim a <code>.lock</code> left behind by a hard kill. Without it, a killed server's lock blocks the file forever.

PARAMETER	DEFAULT	WHAT IT DOES
AutoLockReleaseInterval	<i>none</i>	Milliseconds a lock must exceed in age before it may be reclaimed. Unset means reclaim at any age.
LockReleaseSameNode	<i>true</i>	<code>true</code> reclaims only a lock written by this host — a live node's lock is never stolen. <code>false</code> allows reclaiming another node's.

A lock file holds `random:hostname:hostip:timestamp`. The age is the difference between the current lock's timestamp and the new one's; if it cannot be parsed, the lock is left alone rather than reclaimed on a guess.

AFTER A KILL -9

The symptom is `Couldn't get the lock for processing the file: ...` on every subsequent poll. Set `AutoLockRelease=true` with an interval comfortably longer than a normal poll, or delete the stale `.lock` by hand.

14 · After processing

Names below omit the `transport.vfs.` prefix.

PARAMETER	DEFAULT	WHAT IT DOES
ActionAfterProcess	DELETE	<code>MOVE</code> or <code>DELETE</code> , on success.
MoveAfterProcess	<i>none</i>	Destination folder when the action is <code>MOVE</code> . Required in that case.
ActionAfterFailure	DELETE	<code>MOVE</code> or <code>DELETE</code> , on failure.
MoveAfterFailure	<i>none</i>	Destination folder for failed files.
ActionAfterErrors	<i>inherits</i>	Action for files that errored rather than failed outright.
MoveAfterErrors	<i>inherits</i>	Destination for those.
MoveAfterFailedMove	<i>none</i>	Where to put a file whose move itself failed.
MoveTimestampFormat	<i>none</i>	A date pattern appended to the moved file's name, so repeated runs do not overwrite.
SubFolderTimestampFormat	<i>none</i>	A date pattern used to create a dated sub-folder under the destination.

A file interrupted by a graceful shutdown is **not** moved to the failure folder. It stays in the input folder with its checkpoint and is resumed on the next poll.

A separate failed-record retry family exists (`MaxRetryCount`, `FailedRecordNextRetryDuration`, `FailedRecordsFileName`, `FailedRecordsFileDestination`, `MoveFailedRecordTimestampFormat`, `MoveAfterFailedMove`) and is inherited from the base VFS transport. In streaming mode the per-record error folders above are the ones that matter.

15 · Remote connections

`transport.vfs.FileURI` carries the protocol, the credentials and any VFS options as query parameters.

```
// local
/data/in
file:///data/in

// sftp, with VFS options
sftp://user:pass@host:22/incoming?vfs.passive=true

// ftp / ftps
ftp://user:pass@host/incoming?vfs.passive=true

// smb2
smb2://user:pass@host/share/incoming
```

PROTOCOL	NEEDS
file	Nothing. Shipped with MI.
ftp / ftps	Shipped with MI's commons-vfs.
sftp	Shipped with MI's commons-vfs.
smb2	The <code>smbj</code> and <code>asn-one</code> bundles, present in MI 4.1.0. Requires commons-vfs 2.2.0-wso2v13.20 or later, which carries the sandbox smb2 provider.

Passwords in a `FileURI` are masked in every log line the inbound writes, and never reach the checkpoint.

16 · Context properties

Position properties are stamped on every streamed message, whether the parsed output went to a property or to the body. They let a sequence tell which rows it is holding.

PROPERTY	SET IN	VALUE
STREAMING_RECORD_NUMBER	RECORD	Absolute 1-based record number in the file.
STREAMING_CHUNK_NUMBER	CHUNK	Absolute 1-based chunk number, continuing correctly across a resume.
STREAMING_FIRST_RECORD	both	Absolute number of the first record in this message.
STREAMING_LAST_RECORD	both	Absolute number of the last record in this message.
STREAMING_CHUNK_SIZE	CHUNK	Records actually in this message — smaller than <code>ChunkSize</code> for the final chunk.

Logging progress without a script mediator

These are enough to log a line every 10,000 rows with a plain filter. The XPath is true only on the message that crosses a 10,000 boundary:

```
<filter xpath="floor(number(get-property('STREAMING_LAST_RECORD')) div 10000)
    &gt; floor((number(get-property('STREAMING_FIRST_RECORD')) - 1) div 10000)">
  <then>
    <log level="custom">
      <property name="PROGRESS" value="rows streamed"/>
      <property name="milestone"
        expression="substring-before(concat(floor(number(get-property('STREAMING_LAST_RECORD'))
          div 10000) * 10000, '.0'), '.0')"/>
      <property name="chunk" expression="get-property('STREAMING_CHUNK_NUMBER')"/>
      <property name="rowRange"
        expression="concat(get-property('STREAMING_FIRST_RECORD'), '-',
          get-property('STREAMING_LAST_RECORD'))"/>
    </log>
  </then>
</filter>
```

THE SUBSTRING-BEFORE TRICK

XPath 1.0 arithmetic yields a double, so `floor(...) * 10000` prints as `10000.0`. Concatenating `'.0'` and taking `substring-before(..., '.')` gives a clean integer in both cases.

17 · Four configurations

Each one is complete: the inbound, the sequence, and what comes out.

1 · CSV in chunks to a property, JSON reply file

orders.csv → CHUNK of 2000 → property streamOut → sequence builds a rollup → order-batches.jsonl

The workhorse shape for large files. Nothing of the file is in the message body, so the payload stays small and the sequence reads the chunk from a property.

[inbound-endpoints/ordersInbound.xml](#)

```
<?xml version="1.0" encoding="UTF-8"?>
<inboundEndpoint xmlns="http://ws.apache.org/ns/synapse"
    name="ordersInbound"
    class="org.wso2.carbon.inbound.streaming.VFSConsumer"
    sequence="orderChunkSeq" onError="orderFaultSeq" suspend="false">
  <parameters xmlns="http://ws.apache.org/ns/synapse">
    <parameter name="scheduleType">Polling</parameter>
    <parameter name="interval">5000</parameter>
    <parameter name="sequential">true</parameter>
    <parameter name="coordination">true</parameter>

    <parameter name="transport.vfs.FileURI">/data/in</parameter>
    <parameter name="transport.vfs.FileNamePattern">.*\..csv</parameter>
    <parameter name="transport.vfs.FileProcessCount">1</parameter>
    <parameter name="transport.vfs.Locking">enable</parameter>
    <parameter name="transport.vfs.ActionAfterProcess">MOVE</parameter>
    <parameter name="transport.vfs.MoveAfterProcess">/data/out</parameter>
    <parameter name="transport.vfs.ActionAfterFailure">MOVE</parameter>
    <parameter name="transport.vfs.MoveAfterFailure">/data/fault</parameter>

    <!-- 2000 rows per message, as JSON objects keyed by the header -->
    <parameter name="transport.vfs.Streaming">true</parameter>
    <parameter name="transport.vfs.StreamingMode">CHUNK</parameter>
    <parameter name="transport.vfs.StreamingInputFormat">csv</parameter>
    <parameter name="transport.vfs.StreamingChunkSize">2000</parameter>
    <parameter name="transport.vfs.StreamingBufferSize">65536</parameter>
    <parameter name="transport.vfs.StreamingCsvHasHeader">true</parameter>
    <parameter name="transport.vfs.StreamingAddHeadersToEachResult">true</parameter>
    <parameter name="transport.vfs.StreamingOutputProperty">streamOut</parameter>

    <!-- numbers arrive as JSON numbers so the sequence can do arithmetic -->
    <parameter name="transport.vfs.StreamingCsvDataTypes">[{"Column Name Or Index":"order_id","Is
Column Name":"Yes","Data Type":"Integer"}, {"Column Name Or Index":"unit_price","Is Column
Name":"Yes","Data Type":"Number"}, {"Column Name Or Index":"quantity","Is Column Name":"Yes","Data
Type":"Integer"}]</parameter>

    <parameter name="transport.vfs.ReplyFileURI">/data/replies</parameter>
    <parameter name="transport.vfs.ReplyFileName">order-batches.jsonl</parameter>
  </parameters>
</inboundEndpoint>
```

[sequences/orderChunkSeq.xml](#)

```
<?xml version="1.0" encoding="UTF-8"?>
<sequence name="orderChunkSeq" xmlns="http://ws.apache.org/ns/synapse">
  <script language="js"><![CDATA[
    var rows = JSON.parse(mc.getProperty('streamOut'));
    var total = 0, flagged = [];
    for (var i = 0; i < rows.length; i++) {
      var line = rows[i].unit_price * rows[i].quantity;
      total += line;
      if (line > 10000) { flagged.push(rows[i].order_id); }
    }
    mc.setPayloadJSON({
```

```

        chunk:    mc.getProperty('STREAMING_CHUNK_NUMBER'),
        rows:     rows.length,
        revenue:  Math.round(total * 100) / 100,
        review:   flagged
    });
  ]></script>
</sequence>

```

/data/replies/order-batches.jsonl – one line per chunk

```

{"chunk":"1","rows":2000,"revenue":418233.17,"review":[1041,1388]}
{"chunk":"2","rows":2000,"revenue":402911.80,"review":[]}
{"chunk":"3","rows":1312,"revenue":264550.42,"review":[5120]}

```

2 · CSV row by row into the body, XML reply file

customers.csv → RECORD → message body → payloadFactory → response.xml

With no `StreamingOutputProperty`, the parsed record becomes the body. Because the sequence leaves XML behind, the reply file is written through the normal message formatter.

inbound-endpoints/customerInbound.xml – parameters only

```

<parameter name="scheduleType">Polling</parameter>
<parameter name="interval">5000</parameter>

<parameter name="transport.vfs.FileURI">/data/in</parameter>
<parameter name="transport.vfs.FileNamePattern">customers.*\..csv</parameter>
<parameter name="transport.vfs.ActionAfterProcess">MOVE</parameter>
<parameter name="transport.vfs.MoveAfterProcess">/data/out</parameter>

<parameter name="transport.vfs.Streaming">true</parameter>
<parameter name="transport.vfs.StreamingMode">RECORD</parameter>
<parameter name="transport.vfs.StreamingInputFormat">csv</parameter>
<parameter name="transport.vfs.StreamingCsvHasHeader">true</parameter>
<parameter name="transport.vfs.StreamingAddHeadersToEachResult">true</parameter>
<!-- no StreamingOutputProperty: the record lands in the body -->

<parameter name="transport.vfs.ReplyFileURI">/data/replies</parameter>
<parameter name="transport.vfs.ReplyFileName">customers.xml</parameter>

```

sequences/customerSeq.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<sequence name="customerSeq" xmlns="http://ws.apache.org/ns/synapse">
  <payloadFactory media-type="xml">
    <format><customer row="$1"><id>$2</id><name>$3</name></customer></format>
    <args>
      <arg expression="get-property('STREAMING_RECORD_NUMBER')"/>
      <arg expression="json-eval($.id)"/>
      <arg expression="json-eval($.name)"/>
    </args>
  </payloadFactory>
</sequence>

```

REQUIRED FOR THE BODY PATH WITH CSV

A raw CSV body needs a plain-text message builder, or MI tries to parse it as XML and every record fails with Unexpected character 's' (code 115) in prolog. Add this to `<MI_HOME>/conf/deployment.toml` and restart:

conf/deployment.toml

```
[[custom_message_builders]]
content_type = "text/csv"
class = "org.apache.axis2.format.PlainTextBuilder"
```

3 · Streaming a large JSON array

catalog.json → JsonPath \$.products[*] → RECORD → property item

A JSON document too large to hold whole. The selector picks the repeating element; everything outside it is ignored.

/data/in/catalog.json – the shape being streamed

```
{
  "meta": { "generated": "2026-10-06", "count": 4200000 },
  "products": [
    { "sku": "A-1001", "name": "Hex bolt M8", "price": 0.42, "stock": 18400 },
    { "sku": "A-1002", "name": "Hex bolt M10", "price": 0.55, "stock": 12750 }
  ]
}
```

inbound-endpoints/catalogInbound.xml – parameters only

```
<parameter name="scheduleType">Polling</parameter>
<parameter name="interval">10000</parameter>

<parameter name="transport.vfs.FileURI">/data/in</parameter>
<parameter name="transport.vfs.FileNamePattern">catalog.*.json</parameter>
<parameter name="transport.vfs.ActionAfterProcess">MOVE</parameter>
<parameter name="transport.vfs.MoveAfterProcess">/data/out</parameter>

<parameter name="transport.vfs.Streaming">true</parameter>
<parameter name="transport.vfs.StreamingMode">RECORD</parameter>
<parameter name="transport.vfs.StreamingInputFormat">json</parameter>
<parameter name="transport.vfs.StreamingJsonPath">$.products[*]</parameter>
<parameter name="transport.vfs.StreamingBufferSize">65536</parameter>
<parameter name="transport.vfs.StreamingOutputProperty">item</parameter>

<parameter name="transport.vfs.ReplyFileURI">/data/replies</parameter>
<parameter name="transport.vfs.ReplyFileName">restock.jsonl</parameter>
```

sequences/catalogSeq.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<sequence name="catalogSeq" xmlns="http://ws.apache.org/ns/synapse">
  <property name="sku" expression="json-eval($.sku)" scope="default"/>
  <property name="stock" expression="json-eval($.stock)" scope="default"/>
  <filter xpath="number(get-property('stock')) < 1000">
    <then>
      <payloadFactory media-type="json">
        <format>{"sku":"$1","stock":$2,"record":$3}</format>
        <args>
          <arg expression="get-property('sku')"/>
          <arg expression="get-property('stock')"/>
          <arg expression="get-property('STREAMING_RECORD_NUMBER')"/>
        </args>
      </payloadFactory>
    </then>
    <else><drop/></else>
  </filter>
</sequence>
```

Only the low-stock items reach `restock.jsonl`; the rest are dropped and nothing is appended for them.

4 · Clustered, with checkpointing and completion sequences

two nodes · shared GovRegRoot · resume across a restart · orderCompleteSeq at the end

The full production shape. One node streams; if it stops, the other picks the file up and continues from the last checkpoint rather than from row 1.

inbound-endpoints/bigOrdersInbound.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<inboundEndpoint xmlns="http://ws.apache.org/ns/synapse"
    name="bigOrdersInbound"
    class="org.wso2.carbon.inbound.streaming.VFSConsumer"
    sequence="orderEnrichSeq" onError="orderFaultSeq" suspend="false">
  <parameters xmlns="http://ws.apache.org/ns/synapse">
    <parameter name="scheduleType">Polling</parameter>
    <parameter name="interval">5000</parameter>
    <parameter name="sequential">true</parameter>
    <parameter name="coordination">true</parameter>

    <parameter name="transport.vfs.FileURI">/data/in</parameter>
    <parameter name="transport.vfs.FileNamePattern">.*\..csv</parameter>
    <parameter name="transport.vfs.FileProcessCount">1</parameter>

    <!-- reclaim a .lock left by a hard kill; 30s is well past a normal poll -->
    <parameter name="transport.vfs.Locking">enable</parameter>
    <parameter name="transport.vfs.AutoLockRelease">true</parameter>
    <parameter name="transport.vfs.AutoLockReleaseInterval">30000</parameter>
    <parameter name="transport.vfs.LockReleaseSameNode">false</parameter>

    <parameter name="transport.vfs.ActionAfterProcess">MOVE</parameter>
    <parameter name="transport.vfs.MoveAfterProcess">/data/out</parameter>
    <parameter name="transport.vfs.ActionAfterFailure">MOVE</parameter>
    <parameter name="transport.vfs.MoveAfterFailure">/data/fault</parameter>

    <parameter name="transport.vfs.Streaming">true</parameter>
    <parameter name="transport.vfs.StreamingMode">CHUNK</parameter>
    <parameter name="transport.vfs.StreamingInputFormat">csv</parameter>
    <parameter name="transport.vfs.StreamingChunkSize">2000</parameter>
    <parameter name="transport.vfs.StreamingBufferSize">65536</parameter>
    <parameter name="transport.vfs.StreamingCsvHasHeader">true</parameter>
    <parameter name="transport.vfs.StreamingAddHeadersToEachResult">true</parameter>
    <parameter name="transport.vfs.StreamingOutputProperty">streamOut</parameter>

    <!-- interval is in chunks here, so 50 = a resume point every 100k rows -->
    <parameter name="transport.vfs.StreamingCheckpointEnabled">true</parameter>
    <parameter name="transport.vfs.StreamingCheckpointInterval">50</parameter>

    <parameter name="transport.vfs.StreamingFileCompleteSequence">orderCompleteSeq</parameter>
    <parameter name="transport.vfs.StreamingFileFailureSequence">orderFailedSeq</parameter>

    <parameter name="transport.vfs.StreamingMediationErrorAction">MOVE</parameter>
    <parameter name="transport.vfs.StreamingMediationErrorFolder">/data/fault</parameter>

    <parameter name="transport.vfs.ReplyFileURI">/data/replies</parameter>
    <parameter name="transport.vfs.ReplyFileName">order-batches.jsonl</parameter>
  </parameters>
</inboundEndpoint>
```

sequences/orderCompleteSeq.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<sequence name="orderCompleteSeq" xmlns="http://ws.apache.org/ns/synapse">
  <log level="custom">
    <property name="FILE_DONE" expression="get-property('STREAMING_FILE_NAME')"/>
    <property name="status" expression="get-property('STREAMING_STATUS')"/>
    <property name="processed" expression="get-property('STREAMING_PROCESSED')"/>
    <property name="parseFail" expression="get-property('STREAMING_PARSE_FAILED')"/>
    <property name="mediateFail" expression="get-property('STREAMING_MEDIATION_FAILED')"/>
  </log>
</sequence>
```

```
</log>
</sequence>
```

A verified handoff on this configuration, with a 1 GB file of 8,663,312 rows:

STEP	NODE	RECORDS
File copied in; node1 takes the lock	node1	1 → 2,812,000
node1 killed; checkpoint written	node1	@ 2,812,000
node2, already polling, takes over	node2	2,812,000 → 7,506,000
node2 killed; checkpoint written	node2	@ 7,506,000
node1 restarted, finishes the file	node1	7,506,000 → 8,663,312

The three segments sum to exactly 8,663,312 — no gap and no re-delivery.

18 · Gotchas

- **Both streaming switches are needed.** `transport.vfs.Streaming` defaults to `false`; `StreamingMode` alone does nothing.
- **Jars go in `lib/`, not `dropins/`.** Copy the provided `mi-inbound-streaming-1.0.0.jar` and `commons-csv-1.14.1.jar` into `<MI_HOME>/lib/`, delete any stale `dropins/mi_inbound_streaming_*.jar` so MI regenerates the OSGi bundle, then restart. MI 4.1.0 does not ship `commons-csv`.
- **A raw CSV body needs `PlainTextBuilder` registered for `text/csv` in `deployment.toml`.** Not needed when the output goes to a property.
- **A multi-line `<format>` breaks JSON Lines.** Keep `payloadFactory` formats on one line when writing to a `.jsonl` reply file.
- **Do not symlink the registry** to share it between nodes. Point `GovRegRoot` at the shared path instead.
- **Stopping MI:** the process matches `micro.integrator.bootstrap.Bootstrap`, not `wso2carbon`. A forgotten node that still polls the same folder will compete for files and clear checkpoints out from under the node you are watching.
- **A re-copied file still resumes.** Identity is content, not timestamp, so moving a file out and copying it back does not invalidate its checkpoint.